

Exploring Twitter's - System Design

Designing Twitter is a quite common question that interviewers ask candidates.

A lot of candidates are afraid of this round more than coding round because they don't get an idea of what topics and tradeoffs they should cover within this limited timeframe.

1. How would you design Twitter?

Don't jump into the technical details immediately when you are asked this question in your interview. Do not run in one direction. It will just create confusion between you and the interviewer.

Most of the candidates make mistakes here and immediately they start listing out some bunch of tools/frameworks like MongoDB, Bootstrap, MapReduce etc...

You can put yourself in a situation where you're working on real life projects. Firstly, define the problem and clarify the problem statement.

2. Requirements for Twitter System Design

→ Functional Requirements

- Should be able to post new tweets (can be text, image, video, etc).
- Should be able to follow other users
- Should have a newsfeed feature consisting of tweets from the people the user is following.
- Home timeline ("following" feed) and user profile timeline
- Likes, retweets/reposts, replies/threads.
- Search tweets/users/hashtags
- Notification (Mentions, likes, follows, retweets)

* Trends (top hashtags / topics by region)

→ Non Functional Requirements

- * High Availability for read-heavy Surfaces (home, Profile Search) : $p99 < 300 - 500$ ms.
- * Write Availability for tweet/Engagement : $p99 < 150 - 200$ ms
- * Durability : no lost acknowledged tweets.
- * scalability : Horizontal scale across regions.
- * Consistency : time line Eventually consistent ($< \text{few seconds}$) strong for identity/handles.
- * cost : leverage CDN for media, caching for hot keys, tiered storage for cold Data.

→ Explicit Non-goals (MVP)

- Direct Messages, Audio Spaces, Ads, Recommendations beyond Basic Ranking (note as future work)

3 Capacity Estimation for Twitter System Design

* Traffic Estimation

assume 1 Billion total users with 200 million daily active users, and on average each user tweets 5 times a Day. This give us 1 billion tweets per day

$$200 \text{ Million} * 5 \text{ tweets} = 1 \text{ billion/Day}$$

Tweets can also contains media such as images, or videos. We can assume that 10 percent of tweets are media files shared By Users, which gives us additional 100 million files we would need to store.

$$10 \text{ percent} * 1 \text{ billion} = 100 \text{ million/Day}$$

For our system Request per Second (RPS) will be

1 billion requests per day translate into 12k requests per Second.

$$1 \text{ billion} / (24 \text{ hrs} * 3600 \text{ Seconds}) = 12k \text{ requests/Sec}$$

Throughput

- Tweet Writes : 1B/day $\approx 11,574/s$ (around 12k TPS)
- Home timeline reads : 6B/day $\approx 69,444/s$ ($\sim 70k$ RPS base spikes 5-10x)
- Engagement Writes : (likes / retweets / replies) : assume 3 times tweets $\rightarrow \sim 36k$ TPS.

Storage Estimation

Let's assume each message on average is 100 bytes, we will require about 100 GB of Database storage Everyday

$$1 \text{ billion} * 100 \text{ bytes} = 100 \text{ GB/DAY}$$

10 percent of our daily messages (100 million) are media files per our requirements. Let's assume each file is 50KB on average, we will require 5TB of storage Everyday.

$$100 \text{ million} * 50 \text{ KB} = 5 \text{ TB/day}$$

For 10 years require 19 PB of storage

$$(5 \text{ TB} + 0.1 \text{ TB}) * 365 \text{ days} * 10 \text{ years} = 19 \text{ PB}$$

- Media 15% X 1B X 300KB avg $\approx 45 \text{ TB/day}$ origin ingest
CDN offloads egress.
- Search index (inverted index + term dict + postings) : $\sim 1-2x$ text size
 $\rightarrow \sim 1-3 \text{ TB/day}$.

Bandwidth Estimation

As our system is handling 5.1 TB of ingress Everyday, we will require minimum Bandwidth of around 60MB per Second.

$$5.1 / (24 \text{ hrs} * 3600 \text{ Seconds}) = 60 \text{ MB/Second}$$

Identity vs ID generation

1. Snowflake-style 64-bit IDs (k-sortable, time-ordered)

A 64-bit integer that encodes time + machine identity + per-millisecond sequence, so IDs are *roughly* increasing by time and unique without round-trips to a DB.

- **41 bits** — milliseconds since a chosen epoch (e.g., 2010-11-04).
Range $\approx$ 69 years from the epoch.
- **10 bits** — **node identity** (often split as 5 bits datacenter + 5 bits worker).
Up to 1024 workers globally.
- **12 bits** — **sequence** within the same millisecond.
Up to **4096 IDs/ms/worker** ($\sim 4.1\text{M/sec}$ across 1000 workers).

2. Where Snowflake IDs are used

- **Tweet IDs, Retweet IDs, Notification IDs, Timeline Entry IDs** → fast range scans by time, stable pagination cursors.
- **Shard routing:** For high write locality you'll usually shard tweets by authorId hash, not by tweetId. Use tweetId for lookups and secondary indexes.
- **Search & analytics:** The time component helps build time-windowed indexes and stream processors (late data detection).

3. Assigning worker IDs (10-bit space)

- Static mapping on service boot via a small control plane (e.g., "ID Authority" service) that leases a (datacenterId, workerId) pair with TTL; renew on heartbeat.
- Protect against double assignment (e.g., VM reimage): lease IDs, don't hardcode; on lease loss, stop issuing IDs.

4. Failure & edge cases to plan for

- **Sequence rollover within 1 ms:** wait next ms (most common), or route a portion of traffic to a shadow generator.
- **Clock regression:** refuse to generate (fail fast) vs. "regression sequence" mode; always page oncall it's a production smell.
- **Exhaustion horizon:** 41-bit time runs ~ 69 years from your epoch; document the epoch and set a calendar alert long in advance.

5. User IDs vs. Tweet IDs

- You can also issue userId via Snowflake (good to avoid DB sequences).
- Keep handle and userId separate: handle is mutable (rename), userId is immutable.
- A dedicated "handles" table maps handle → userId with a strict unique constraint; a secondary mapping userId → currentHandle helps joins and backfills.

Tomorrow I will try to explore Twilio system
Design Mode.